3_Excepciones

September 21, 2026

Contenido creado por: Isabel Maniega

---

1 -1- Excepciones: en la web de Python

[1]:
```
# https://docs.python.org/3/library/exceptions.html
```

---

2 -2- Ejemplo básico try-except

[2]:
```
# Imagina que tenemos 2 variables
```

[3]:
```
x = 5
# 'y' no la tenemos definida
```

[4]:
```
print(x)
```

```
5
```

[5]:
```
# print(y)
# NameError: name 'y' is not defined
# OBVIAMENTE DA ERROR, AL NO TENERLA DEFINIDA
```

[6]:
```
try:
    print(x)
except:
    print("No tenemos definida la variable:")
```

```
5
```

[7]:
```
try:
    print(y)
except:
    print("No tenemos definida la variable:")
```

```
No tenemos definida la variable:
```

[8]:
```
# De esta forma podemos conseguir que un código funcione saltando un error.

# Pero OJO! en donde colocamos este Try-Except. Porque si es algo crítico␣
 ↪estamos creando un problema.

# Solo sirve cuando es algo que necesitamos saltar,

# (para que el código ejecute en un momento que sabemos que algo no va)
```

---

3 -3- Forma básica de crear una excepción

3.1 -3.1- En la División por cero

[9]:
```
a = 5
b = 0
# a/b
# Si descomentamos "a/b" nos sale:
# ZeroDivisionError: division by zero

# No es posible dividir un número por cero. (Daria infinito)
```

Podemos hacer lo siguiente, sin excepciones

[10]:
```
def funcion_dividir_1(a, b):
    if b != 0:
        print(a / b)
    else:    # b = 0  -> no puede dividir
        print("el denominador es 0, no podemos dividir")
```

[11]:
```
funcion_dividir_1(2,3)
```

```
0.6666666666666666
```

[12]:
```
funcion_dividir_1(2,0)
```

```
el denominador es 0, no podemos dividir
```

el mismo ejercicio cambiando el operador

[13]:
```
def funcion_dividir_1(a, b):
    if b == 0:
        print("el denominador es 0, no podemos dividir")
    else:      # b es distinto de 0
        print(a/b)
```

[14]:
```
funcion_dividir_1(2,3)
```

```
0.6666666666666666
```

[15]:
```
funcion_dividir_1(2,0)
```

```
el denominador es 0, no podemos dividir
```

---

4 -4- Uso de raise

[16]:
```
# Podemos lanzar excepciones, no lo usaremos
```

---

5 -5- Try-Except-Else

[17]:
```
# me creo una función para comprobar más casos.

def funcion_division(a,b):
    try:
        division = a / b
        print('estamos en try y hemos calculado a/b')
    except ZeroDivisionError:
        print("Un número dividido por 0 sale infinito")
        print("No pongas un 0 en el deniminador!")
    else:
        print('estamos en el else')
        print('valor de la división:', division)
```

[18]:
```
funcion_division(1,0)
```

```
Un número dividido por 0 sale infinito
No pongas un 0 en el deniminador!
```

[19]:
```
funcion_division(1,2)
```

```
estamos en try y hemos calculado a/b
estamos en el else
valor de la división: 0.5
```

---

6 -6- try-except-else con archivos

Un archivo que no existe

(o no se encuentra en ese lugar)

[20]:
```
try:
    f = open('archivo_excepciones.txt')  # El fichero no existe
except FileNotFoundError:
```

```
    print('¡El fichero no existe!')
else:
    print(f.read())
```

```
¡El fichero no existe!
```

[21]:
```
# lo cerramos, si esta abierto
# f.close()
```

Un archivo que SI existe

(y lo encuentra en esa ubicación)

[22]:
```
try:
    f = open('archivo_excepciones_2.txt')  # El fichero si existe
except FileNotFoundError:
    print('¡El fichero no existe!')
else:
    print(f.read())
    f.close()
```

```
Hola Mundo!!

Bienvenidos a este archivo.
```

---

7 -7- Errores cuando sumamos strings en vez de números

[23]:
```
# print(2 + "2")
# TypeError: unsupported operand type(s) for +: 'int' and 'str'

# al hacer esa operación nos devuelve un error
```

[24]:
```
3
```

[24]:
```
3
```

[25]:
```
type(3)
```

[25]:
```
int
```

[26]:
```
str(3)
```

[26]:
```
'3'
```

[27]:
```
'3'
```

[27]:
```
'3'
```

[28]:
```
type('3')
```

[28]:
```
str
```

[29]:
```
def funcion_formatos_diferentes(a,b):
    try:
        suma = a + b
        print(suma)
    except TypeError:
        print("revisa el formato de los números, porque no es correcto")
```

[30]:
```
funcion_formatos_diferentes(2,"3")
```

```
revisa el formato de los números, porque no es correcto
```

[31]:
```
funcion_formatos_diferentes(2,3)
```

```
5
```

---

8 -8- except Exception

Otra forma si no sabes que excepción puede saltar,

puedes usar la clase genérica Exception.

Sirve para cualquier tipo de excepción.

De hecho todas las excepciones heredan de Exception **except Exception: Ejemplo 1**

[32]:
```
def funcion_suma_2(a,b):
    try:
        suma = a + b
        print("la suma es: ", suma)
    except Exception:
        print("Ha habido una excepción")
```

[33]:
```
funcion_suma_2(2,0)
```

```
la suma es:  2
```

[34]:
```
funcion_suma_2(2,"2")
```

```
Ha habido una excepción
```

except Exception: Ejemplo 2

[35]:
```
def funcion_division_3(a,b):
    try:
        division = a / b
        print("la division es: ", division)
```

```
    except Exception:
        print("Ha habido una excepción")
```

[36]:
```
funcion_division_3(1,3)
```

```
la division es:  0.3333333333333333
```

[37]:
```
funcion_division_3(2,0)
```

```
Ha habido una excepción
```

[38]:
```
funcion_division_3(2,"2")
```

```
Ha habido una excepción
```

---

9 -9- except Exception as e (una de las mejores opciones)

[39]:
```
def funcion_division_4(a,b):
    try:
        division = a / b
        print("la division es: ", division)
    except Exception as e:
        print("Ha habido una excepción")
        print("tipo del error: ", type(e))
        print('str(e):', str(e))
```

[40]:
```
funcion_division_4(1,3)
```

```
la division es:  0.3333333333333333
```

[41]:
```
funcion_division_4(2,0)
```

```
Ha habido una excepción
tipo del error:  <class 'ZeroDivisionError'>
str(e): division by zero
```

[42]:
```
funcion_division_4(2,"2")
```

```
Ha habido una excepción
tipo del error:  <class 'TypeError'>
str(e): unsupported operand type(s) for /: 'int' and 'str'
```

---

10 -10- except Exception as e (otra posibilidad: try-except-else)

[43]:
```
def funcion_division_5(a,b):
    try:
        division = a / b
        print("la division es: ", division)
    except Exception as e:
        print("Ha habido una excepción")
        print("tipo del error: ", type(e))
    else:
        print("estamos en else, no hubo excepciones")
```

[44]:
```
funcion_division_5(1,3)
```

```
la division es:  0.3333333333333333
estamos en else, no hubo excepciones
```

[45]:
```
funcion_division_5(2,0)
```

```
Ha habido una excepción
tipo del error:  <class 'ZeroDivisionError'>
```

[46]:
```
funcion_division_5(2,"2")
```

```
Ha habido una excepción
tipo del error:  <class 'TypeError'>
```

---

11 -11- except Exception as e (otra posibilidad: try-except-finally)

Este bloque se suele usar si queremos ejecutar algún tipo de acción de limpieza.

Si por ejemplo estamos escribiendo datos en un fichero pero ocurre una excepción,

tal vez queramos borrar el contenido que hemos escrito con anterioridad,

para no dejar datos inconsistenes en el fichero.

[47]:
```
def funcion_division_6(a,b):
    try:
        division = a / b
        print("la division es: ", division)
        print("\n")
    except Exception as e:
        print("Ha habido una excepción")
        print("tipo del error: ", type(e))
        print("\n")
    finally:
        print("estamos en finally")
```

```
        print("esto se ejecuta SIEMPRE haya o no excepciones")
```

[48]:
```
funcion_division_6(1,3)
```

```
la division es:  0.3333333333333333


estamos en finally
esto se ejecuta SIEMPRE haya o no excepciones
```

[49]:
```
funcion_division_6(2,0)
```

```
Ha habido una excepción
tipo del error:  <class 'ZeroDivisionError'>


estamos en finally
esto se ejecuta SIEMPRE haya o no excepciones
```

[50]:
```
funcion_division_6(2,"2")
```

```
Ha habido una excepción
tipo del error:  <class 'TypeError'>


estamos en finally
esto se ejecuta SIEMPRE haya o no excepciones
```

---

12 -12- Ejemplo de excepciones con archivos

[51]:
```
def funcion_lectura(archivo):
    try:
        with open(archivo) as file:
            lectura_archivo = file.read()
            print(lectura_archivo)
    except Exception as e:
        print("no se pudo abrir")
        print("Tipo de error:", type(e))
        print(str(e))
```

[52]:
```
funcion_lectura('archivo_excepciones_1.txt')  # no lo encuentra
```

```
no se pudo abrir
Tipo de error: <class 'FileNotFoundError'>
[Errno 2] No such file or directory: 'archivo_excepciones_1.txt'
```

[53]:
```
funcion_lectura('archivo_excepciones_2.txt') # si lo encuentra

# (si lo coloco yo previamente este archivo)
# SE ENCUENTRA EN LA MISMA RUTA
```

```
Hola Mundo!!

Bienvenidos a este archivo.
```

---

13 Más sobre Excepciones...

Ordenadas las excepciones por orden de preferencia:

```
BaseException
  BaseExceptionGroup
  GeneratorExit
  KeyboardInterrupt
  SystemExit
  Exception
    ArithmeticError
         FloatingPointError
         OverflowError
         ZeroDivisionError
    AssertionError
    AttributeError
    BufferError
    EOFError
    ExceptionGroup [BaseExceptionGroup]
    ImportError
         ModuleNotFoundError
    LookupError
         IndexError
         KeyError
    MemoryError
    NameError
         UnboundLocalError
    OSError
         BlockingIOError
         ChildProcessError
         ConnectionError
              BrokenPipeError
              ConnectionAbortedError
              ConnectionRefusedError
              ConnectionResetError
         FileExistsError
         FileNotFoundError
```

```
        InterruptedError
        IsADirectoryError
        NotADirectoryError
        PermissionError
        ProcessLookupError
        TimeoutError
    ReferenceError
    RuntimeError
        NotImplementedError
        RecursionError
    StopAsyncIteration
    StopIteration
    SyntaxError
        IndentationError
            TabError
    SystemError
    TypeError
    ValueError
        UnicodeError
            UnicodeDecodeError
            UnicodeEncodeError
            UnicodeTranslateError
    Warning
         BytesWarning
         DeprecationWarning
         EncodingWarning
         FutureWarning
         ImportWarning
         PendingDeprecationWarning
         ResourceWarning
         RuntimeWarning
         SyntaxWarning
         UnicodeWarning
         UserWarning
```

13.1 Tipos de excepciones más relevantes

Para capturar cualquier excepción podemos usar Exception o BaseException:

[54]:
```
try:
    30* (2/0)
except BaseException as e:
    print('Error %s' % str(e))
```

```
Error division by zero
```

[55]:
```
try:
    30* (2/0)
```

```
except Exception as e:
    print('Error %s' % str(e))
```

```
Error division by zero
```

13.1.1 ArithmeticError

- Division entre 0: **ZeroDivisionError**

[56]:
```
30 * (2/0)
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[56], line 1
----> 1 30 * (2/0)

ZeroDivisionError: division by zero
```

[57]:
```
try:
    30* (2/0)
except ZeroDivisionError as e:
    print('Error --> %s' % str(e))
```

```
Error --> division by zero
```

13.1.2 AttributeError

- Error en el uso: **AttributeError**

[58]:
```
num = 10
num.append(6)
print(num)
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[58], line 2
      1 num = 10
----> 2 num.append(6)
      3 print(num)

AttributeError: 'int' object has no attribute 'append'
```

[59]:
```
try:
    num= 10
    num.append(6)
    print(num)
except AttributeError as e:
```

```
    print('Error %s' % str(e))
```

```
Error 'int' object has no attribute 'append'
```

13.1.3 ImportError

- Error al importar un módulo: **ImportError**

[60]:
```
from pandas import hola
```

```
---------------------------------------------------------------------------
ImportError                               Traceback (most recent call last)
Cell In[60], line 1
----> 1 from pandas import hola

ImportError: cannot import name 'hola' from 'pandas' (/home/isabelmaniega/
 ↪Documentos/Programacion Python 20h/venv/lib/python3.12/site-packages/pandas/
 ↪__init__.py)
```

[61]:
```
try:
    from pandas import hola
except ImportError as e:
    print('Error %s' % str(e))
```

```
Error cannot import name 'hola' from 'pandas'
(/home/isabelmaniega/Documentos/Programacion Python
20h/venv/lib/python3.12/site-packages/pandas/__init__.py)
```

Error en importar un modulo será: **ModuleNotFoundError**

[62]:
```
try:
    import hola
except ModuleNotFoundError as e:
    print('Error %s' % str(e))
```

```
Error No module named 'hola'
```

13.1.4 LookupError

- Error de índice: **IndexError**

[63]:
```
L = [10, 50, 60]
L[3]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[63], line 2
      1 L = [10, 50, 60]
----> 2 L[3]
```

```
IndexError: list index out of range
```

[64]:
```
try:
    L = [10, 50, 60]
    L[3]
except IndexError as e:
    print('Error %s' % str(e))
```

```
Error list index out of range
```

Si en vez de poner un número entero ponemos un string el error sería de tipo:

[65]:
```
try:
    L = [10, 50, 60]
    L['3']
except IndexError as e:
    print('Error Index %s' % str(e))
except TypeError as e:
    print('Error TypeError %s' % str(e))
```

```
Error TypeError list indices must be integers or slices, not str
```

- Error de clave en un diccionario: **KeyError**

[66]:
```
ages = {'Juan': 25, 'Luis':36, 'Pedro':41}
ages['Maria']
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[66], line 2
      1 ages = {'Juan': 25, 'Luis':36, 'Pedro':41}
----> 2 ages['Maria']

KeyError: 'Maria'
```

[67]:
```
try:
    ages = {'Juan': 25, 'Luis':36, 'Pedro':41}
    ages['Maria']
except KeyError as e:
    print('Error %s' % str(e))
```

```
Error 'Maria'
```

13.1.5 NameError

- Nombre no definido: **NameError**

[68]:
```
4 + w*3
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[68], line 1
----> 1 4 + w*3

NameError: name 'w' is not defined
```

[69]:
```
try:
    4 + w*3
except NameError as e:
    print('Error %s' % str(e))
```

```
Error name 'w' is not defined
```

13.1.6 OSError

- Archivo no encontrado: **FileNotFoundError**

[70]:
```
try:
    file = open('data.csv')
except FileNotFoundError as e:
    print('Error %s' % str(e))
```

```
Error [Errno 2] No such file or directory: 'data.csv'
```

- Archivo no encontrado: **IsADirectoryError**

[71]:
```
try:
    # crear una carpeta vacia llamada "solucion" al lado del archivo␣
 ↪Excepciones.ipynb
    file = open('./solucion')
except IsADirectoryError as e:
    print('Error %s' % str(e))
```

```
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last)
Cell In[71], line 4
      1 try:
      2     # crear una carpeta vacia llamada "solucion" al lado del archivo␣
 ↪Excepciones.ipynb
      3     file = open('./solucion')
----> 4 except IsADirectoryError as e:
      5     print('Error %s' % str(e))

FileNotFoundError: [Errno 2] No such file or directory: './solucion'
```

13.1.7 SyntaxError

- Error en la identación o sintaxis: **SyntaxError**

IndentationError

[72]:
```
name = 'Pepe'

if name == 'Pepe':
print('El nombre es Pepe')
```

```
  Cell In[72], line 4
    print('El nombre es Pepe')
    ^
IndentationError: expected an indented block after 'if' statement on line 3
```

[73]:
```
name = 'Pepe'

try:
    if name == 'Pepe':
    print('El nombre es Pepe')
except IndentationError as e:
    print('Error %s' % str(e))
```

```
  Cell In[73], line 5
    print('El nombre es Pepe')
    ^
IndentationError: expected an indented block after 'if' statement on line 4
```

Este no se puede capturar, solo si se realiza con dos scripts y se importa uno en otro, podremos realizar la excepción

[74]:
```
# test1.py
try:
    import test2
except IndentationError as ex:
    print(ex)

# test2.py
def f():
    pass
        pass # error
```

```
  Cell In[74], line 10
    pass # error
```

```
    ^
IndentationError: unexpected indent
```

SyntaxError

Por ejemplo si definimos mal un string, lista, etc se nos olvida el cierre

[75]:
```
name = 'Pepe
```

```
  Cell In[75], line 1
    name = 'Pepe
           ^
SyntaxError: unterminated string literal (detected at line 1)
```

[76]:
```
try:
    name = 'Pepe
except SyntaxError as e:
    print('Error %s' str(e))
```

```
  Cell In[76], line 2
    name = 'Pepe
           ^
SyntaxError: unterminated string literal (detected at line 2)
```

Pasa lo mismo que con la identación, no se puede capturar.

13.1.8 TypeError

- Error de tipo de variable: **TypeError**

[77]:
```
'4' + 2
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[77], line 1
----> 1 '4' + 2

TypeError: can only concatenate str (not "int") to str
```

[78]:
```
try:
    '4' + 2
except TypeError as e:
```

```
    print('Error %s' % str(e))
```

```
Error can only concatenate str (not "int") to str
```

13.1.9 ValueError

- Error al recibir un error de tipo o de valor inapropiado: **ValueError**

[79]:
```
import math

x = -3

print(f'Square Root of {x} is {math.sqrt(x)}')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[79], line 5
      1 import math
      2
      3 x = -3
      4
----> 5 print(f'Square Root of {x} is {math.sqrt(x)}')

ValueError: math domain error
```

[80]:
```
x = -3

try:
    print(f'Square Root of {x} is {math.sqrt(x)}')
except ValueError as ve:
    print(f'You entered {x}, which is not a positive number.')
    print('Error %s' % str(ve))
```

```
You entered -3, which is not a positive number.
Error math domain error
```

13.1.10 Múltiples excepciones

En el caso de declarar multiples excepciones se tomarán por orden de preferencia según la primera tabla al ser detectados, para declararlos se pone except y entre paréntesis las excepciones:

[81]:
```
try:
    # Error al pulsar enter sin insertar dato o insertar un texto, error de␣
 ↪TypeError
    value = input('Inserte un número: ')
    result = 25/int(value)

    print(f'El resultado es: {result}')
```

```
    print(f'Square Root of {x} is {math.sqrt(int(value))}')


    L = [10, 5, 6]
    print(f'El valor en la lista es: {L[value]}')
except (IndexError, TypeError, ValueError) as e:
    print('Error %s' % str(e))
```

```
Inserte un número:
```

```
Error invalid literal for int() with base 10: ''
```

[85]:
```
try:
    # Error al poner un número. el index es un string
    value = input('Inserte un número: ')
    result = 25/int(value)
    print(f'El resultado es: {result}')
    print(f'Square Root of {x} is {math.sqrt(int(value))}')
    L = [10, 5, 6]
    print(f'El valor en la lista es: {L[value]}')
except (IndexError, TypeError, ValueError) as e:
    print('Error %s' % str(e))
```

```
Inserte un número:  2
```

```
El resultado es: 12.5
Square Root of -3 is 1.4142135623730951
Error list indices must be integers or slices, not str
```

[83]:
```
try:
    # Error al poner un número negativo
    value = input('Inserte un número: ')
    result = 25/int(value)
    print(f'El resultado es: {result}')
    print(f'Square Root of {x} is {math.sqrt(int(value))}')
    L = [10, 5, 6]
    print(f'El valor en la lista es: {L[value]}')
except (IndexError, TypeError, ValueError) as e:
    print('Error %s' % str(e))
```

```
Inserte un número:  -2
```

```
El resultado es: -12.5
Error math domain error
```

13.1.11 Raise

También se puede usar raise directamente con las excepciones:

[86]:
```
# Error al poner un número un número negativo
value = input('Inserte un número: ')

if not type(value) is int:
    raise TypeError('Error en el index')
```

```
Inserte un número:  -2
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[86], line 5
      1 # Error al poner un número un número negativo
      2 value = input('Inserte un número: ')
      3
      4 if not type(value) is int:
----> 5     raise TypeError('Error en el index')

TypeError: Error en el index
```

---

14 -14- EJERCICIOS

Ejemplo con try except

(el primero es el de examen)

[87]:
```
"""
try:
    print(5/0)
    break
except:
    print("Sorry, something went wrong...")
except (ValueError, ZeroDivisionError):
    print("Too bad...")
"""

# SyntaxError: 'break' outside loop
```

[87]:
```
'\ntry:\n    print(5/0)\n    break\nexcept:\n    print("Sorry, something went
wrong…")\nexcept (ValueError, ZeroDivisionError):\n    print("Too bad…")\n'
```

[88]:
```
# ejemplo 2 de este tipo (este si funciona)

# se ha intentado que ejecute la parte de ZeroDivisionError
```

[89]:
```
try:
    print(5/0)
except (ValueError, ZeroDivisionError):
    print("Too bad...")
except:
    print("Sorry, something went wrong...")
```

```
Too bad...
```

[90]:
```
try:
    print(5/0)
except (ValueError):
    print("Too bad...")
except:
    print("Sorry, something went wrong...")
```

```
Sorry, something went wrong...
```

[91]:
```
try:
    print(5/0)
except ValueError:
    print("Too bad...")
except:
    print("Sorry, something went wrong...")
```

```
Sorry, something went wrong...
```

UNA POSIBILIDAD

[92]:
```
try:
    print(5/0)
except Exception as e:
    print(type(e))
    print(str(e))
```

```
<class 'ZeroDivisionError'>
division by zero
```

[93]:
```
# try:
#     print(5/0)
# except:
#     print("Sorry, something went wrong...")
# except (ValueError, ZeroDivisionError):
#     print("Too bad...")

# SyntaxError: default 'except:' must be last
```

Gracias por la atención